College of Computer Science and Information Technology
Department of Computer Engineering

ARTI 404 – Image Processing

Lab 7(A) Report: Laplacian Filter

Student Name Mohammed Omar Halawani

Student ID 2240001174

Academic Year 2025 – 2026

1. Objective

This lab uses the Laplacian, a second-derivative filter, to find the edges of an image and add them back to the original to make it look sharper. The Laplacian is applied in two ways: with a manually defined 3×3 kernel and `cv2.filter2D`, and with OpenCV's built-in `cv2.Laplacian` after a Gaussian blur. Everything was implemented in Python with OpenCV, NumPy and Matplotlib.

2. Image Used

- **moon.jpg** – 401 × 438 image of the moon, stored as a 3-channel JPEG and loaded as grayscale. It has many small craters, a sharp bright rim on the lower left and a black background.

3. Setup

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

4. Task 1 – Laplacian with a Manual Kernel

The Laplacian of an image is $\nabla^2 f = \partial^2 f/\partial x^2 + \partial^2 f/\partial y^2$. Using finite differences in x and y gives the 4-neighbour kernel below, whose weights sum to zero:

$$[\,0\;1\;0\,;\,1\;-4\;1\,;\,0\;1\;0\,]$$

The kernel is applied with `cv2.filter2D` using the output depth `cv2.CV_64F`, because the Laplacian has both negative and positive values that an 8-bit output would clip. `cv2.convertScaleAbs` then takes the absolute value and converts it back to 8-bit, and the result is added to the original image.

```
# ================ ( Laplacian Filter ) ===================

# 0. Image file path
imgFile = "moon.jpg"

# 1. Load image and convert to grayscale
img = cv2.imread(imgFile, 0)

# 2. Manually define the Laplacian Kernel
# This is the standard 3x3 kernel that detects horizontal and vertical edges
manual_kernel = np.array([
    [0,  1,  0],
```

```
    [1, -4,  1],
    [0,  1,  0]
], dtype=np.float32)

# Alternative kernel: Detects diagonal edges as well
# manual_kernel = np.array([
#     [1,  1,  1],
#     [1, -8,  1],
#     [1,  1,  1]
# ], dtype=np.float32)

# 3. Apply the filter using filter2D
# -1 indicates the output image will have the same depth as the source
# However, for Laplacian, use cv2.CV_64F to avoid clipping negative values
laplacian_manual = cv2.filter2D(img, cv2.CV_64F, manual_kernel)

# 4. Take absolute value and convert back to 8-bit (0-255)
laplacian_abs = cv2.convertScaleAbs(laplacian_manual)

finalImage = img + laplacian_abs

# 5. Display images
plt.figure(figsize=(12, 10))
plt.subplot(221), plt.imshow(img, cmap='gray'), plt.title('Original Image')
plt.subplot(222), plt.imshow(laplacian_manual, cmap='gray'), plt.title('Laplacian Image')
plt.subplot(223), plt.imshow(laplacian_abs, cmap='gray'), plt.title('Convert with Abs
Scale')
plt.subplot(224), plt.imshow(finalImage, cmap='gray'), plt.title('Final Image')
plt.show()
```

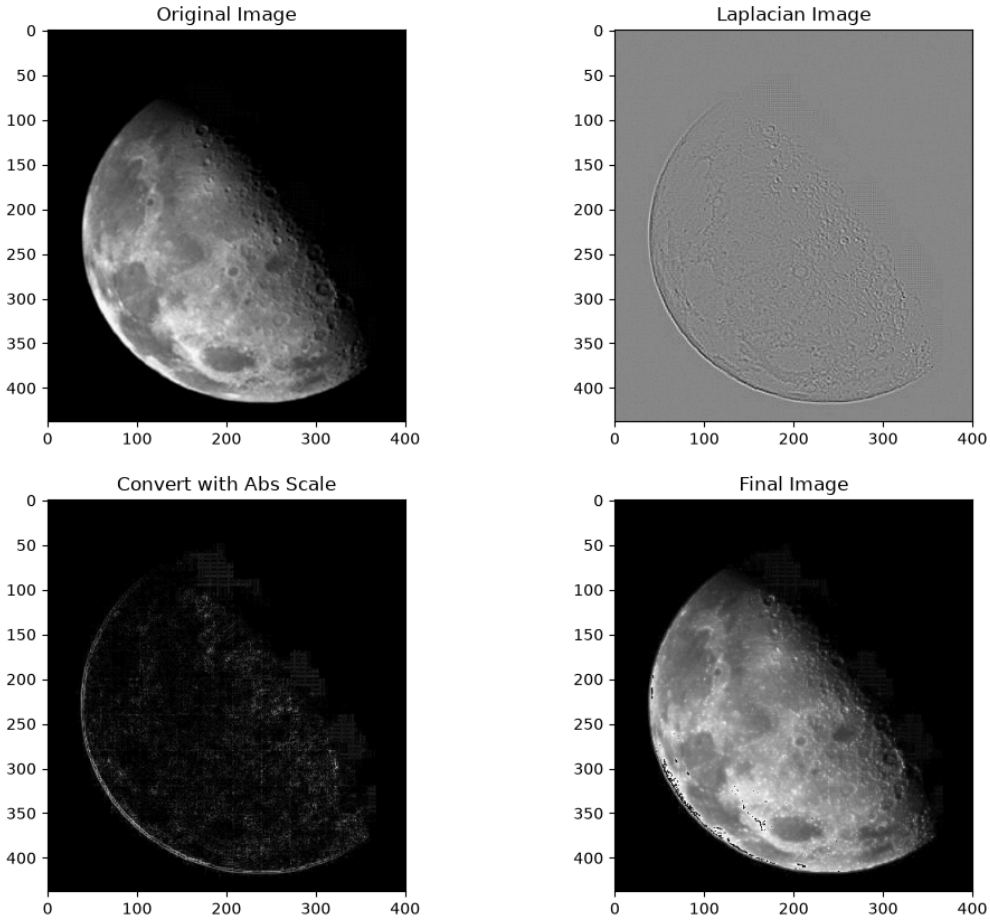


Figure 1: Original image, signed Laplacian, absolute Laplacian and the final image (original + |Laplacian|) using the manual 4-neighbour kernel.

Discussion: The signed Laplacian ranges from about −123 to 110. In the plot, flat regions such as the black sky and the smooth maria are mid-gray, while the crater rims and the outer rim of the moon show up as pairs of bright and dark lines. The absolute image shows the same edges as bright lines on black, but it no longer shows which side of the edge is brighter. The Laplacian also brings out a faint square block pattern in the dark area on the upper right, which most likely comes from JPEG compression. In the final image, the craters and the rim are slightly more pronounced than in the original. However, two details of the code affect the result:

- **uint8 overflow:** `img` and `laplacian_abs` are both `uint8`, so NumPy adds them modulo 256. 541 pixels (about 0.3% of the image) go above 255 and wrap around to dark values. They are the black specks along the bright lower-left rim of the final image. Converting to a larger type and clipping, for example with `cv2.add`, would avoid this.
- **Absolute value instead of sign:** Because the centre coefficient is negative, the usual sharpening formula is $g = f - \nabla^2 f$, which brightens the bright side of an edge and darkens the dark side. Adding $|\nabla^2 f|$ brightens *both* sides, so the edges become brighter and slightly thicker instead of gaining contrast.

The commented-out 8-neighbour kernel [1 1 1; 1 −8 1; 1 1 1] would also respond to diagonal edges and give a somewhat stronger result.

5. Task 2 – Gaussian Blur and cv2.Laplacian

The image is first smoothed with a 3×3 Gaussian so that noise and tiny details are not picked up as edges. Then `cv2.Laplacian` is applied with the default `ksize=1`, which uses exactly the same 4-neighbour kernel as Task 1 (the outputs on the same input are identical). The absolute value is added to the original, unblurred image in the same way.

```
# ================== ( Laplacian Filter ) ===============

# 1. Optional: Apply Gaussian Blur to reduce noise
# This prevents the algorithm from picking up tiny details as edges
blurred = cv2.GaussianBlur(img, (3, 3), 0)

# 2. Apply the Laplacian Algorithm
# Use CV_64F to capture both positive and negative gradients (essential for edges)
laplacian = cv2.Laplacian(blurred, cv2.CV_64F)

# 3. Convert back to 8-bit image to view it properly
laplacian_8bit = cv2.convertScaleAbs(laplacian)

# 4. Adding Laplace and origianl Image
finalImage2 = img + laplacian_8bit

# 5. Display images
plt.figure(figsize=(12, 10))
plt.subplot(221), plt.imshow(img, cmap='gray'), plt.title('Original Image')
plt.subplot(222), plt.imshow(laplacian, cmap='gray'), plt.title('Laplacian Image')
plt.subplot(223), plt.imshow(laplacian_8bit, cmap='gray'), plt.title('Convert with Abs
Scale')
plt.subplot(224), plt.imshow(finalImage2, cmap='gray'), plt.title('Final Image')
plt.show()
```

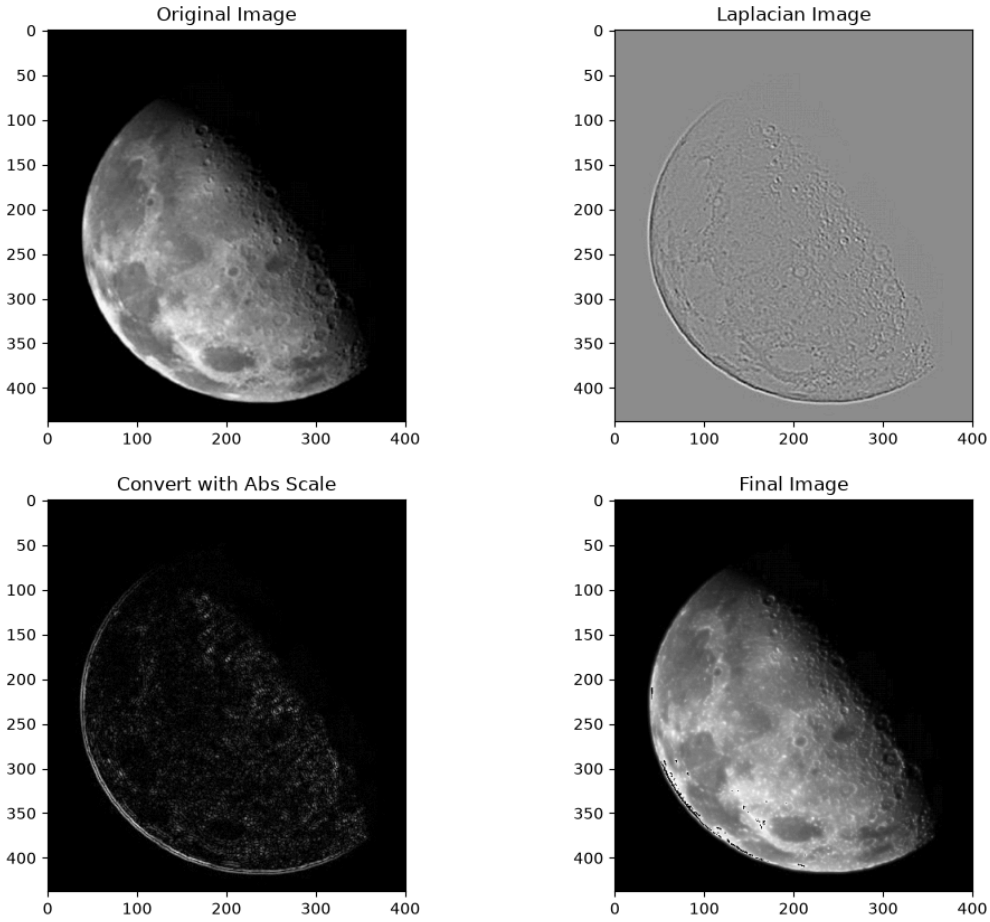


Figure 2: Original image, Laplacian of the Gaussian-blurred image, its absolute value and the final image.

Discussion: Blurring first is a simple form of the Laplacian of Gaussian (LoG). The response becomes weaker and cleaner: the range shrinks to about −66 to 53, and the mean absolute value falls from 4.1 to 2.3. The fine grain and the JPEG block pattern almost disappear from the Laplacian image, while the crater rims and the outer rim remain. Since the blur spreads each edge over more pixels, the rim of the moon now shows up as a wider double line. The final image looks smoother and less noisy than in Task 1, with a smaller sharpening effect. The same uint8 overflow still happens, but at fewer pixels (395), and the black specks along the rim are still visible.

6. Conclusion

The Laplacian is a second-derivative filter: it is zero in flat regions and large at edges, so it is a natural tool for edge detection and sharpening. Implementing it with a manual kernel and `cv2.filter2D` gives the same result as `cv2.Laplacian`. Its main weakness is sensitivity to noise and compression artifacts, and smoothing the image with a Gaussian first gives a cleaner edge map with less sharpening. The way the result is combined with the original also matters: the computation should be done in a larger data type before converting back to 8-bit, and the signed Laplacian should be subtracted ($g = f - \nabla^2 f$)

instead of its absolute value being added. Otherwise the result contains wrap-around artifacts and edges that are brightened rather than sharpened.

7. References

R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, Pearson.

Sandipan Dey, *Hands-On Image Processing with Python*, Packt.

OpenCV: Image Gradients (Laplacian) – docs.opencv.org/4.x/d5/d0f/tutorial_py_gradients.html